

PRÁCTICA N.º 2 — ENUNCIADO

Docker: Imágenes, Contenedores, Volúmenes, Redes y Docker Compose

COM-600 · Microservicios · Universidad San Francisco Xavier

Datos de la práctica	
Asignatura	COM-600 · Microservicios
Tema	Tema 3 — Contenedores y Docker
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 3 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)

Nota: Este documento es el ENUNCIADO: contiene los comandos a ejecutar, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario "Hoja de Evidencias", que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Diferenciar los conceptos de imagen, contenedor, capa, volumen y red en Docker.
- Operar el ciclo de vida completo de un contenedor desde la línea de comandos.
- Escribir un Dockerfile y construir imágenes propias, aplicando buenas prácticas de cacheo de capas y reducción de tamaño.
- Persistir datos con volúmenes nombrados y usar bind mounts para desarrollo.
- Crear redes definidas por el usuario y comunicar contenedores por nombre de servicio (DNS interno).
- Orquestrar una aplicación multi-contenedor (API Node.js + MongoDB) con Docker Compose.
- Aplicar el enfoque de contenedores como base tecnológica de una arquitectura de microservicios.

2. Requisitos previos

Requisito	Verificación	Observación
Docker Desktop 4.x o Docker Engine 24+	<code>docker --version</code>	En Windows debe estar activo WSL2
Docker Compose v2	<code>docker compose version</code>	Se escribe sin guion: <code>docker compose</code>
Editor de código	VS Code recomendado	Extensión "Docker" de Microsoft
Cliente HTTP	Postman / Thunder Client / curl	Para probar la API
Cuenta en Docker Hub	hub.docker.com	Necesaria para el Laboratorio 6

Requisito	Verificación	Observación
Git	<code>git --version</code>	Para subir la entrega al repositorio

Atención: Antes de empezar cree una carpeta de trabajo llamada `practica2-docker` y realice TODO el trabajo dentro de ella. Al final esa carpeta se sube al repositorio.

Los comandos se muestran precedidos por el símbolo `$`. NO escriba el símbolo `$` al copiar el comando.

3. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 20 capturas y se responden las 7 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guiada	7 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables:

- Hoja de Evidencias **en PDF**, con las 20 capturas pegadas y las 7 preguntas de comprobación respondidas (Parte 1).
- Repositorio Git **público** con la carpeta `practica2-docker`: código, `Dockerfile`, `.dockerignore`, `docker-compose.yml`, `.env.example` y `README.md`.
- Informe PDF de la Parte 2** con la resolución de cada ejercicio, capturas de la evidencia y las respuestas a las preguntas de reflexión.

Nota: El archivo `.env` real NO se sube al repositorio. Se sube `.env.example` con las variables sin valores sensibles, y `.env` se agrega al `.gitignore`. Esta es una práctica estándar en despliegue de microservicios.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de "Captura N.º", tome la captura de pantalla de la terminal (o del navegador, según se indique) y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las "Pregunta N.º" se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Tip: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y preguntas están numeradas igual en ambos.

Laboratorio 0 — Verificación del entorno

Objetivo: confirmar que Docker está instalado y que el demonio (Docker Engine) está corriendo.

Paso 1. Verifique la versión del cliente y del motor

```
$ docker --version
$ docker compose version
$ docker info
```

El comando `docker info` muestra el estado del demonio: número de contenedores, imágenes, driver de almacenamiento y sistema operativo del host. Si devuelve un error de conexión, Docker Desktop no está iniciado.

Paso 2. Ejecute el contenedor de prueba

```
$ docker run hello-world
```

Este comando hace tres cosas de golpe: busca la imagen en local, no la encuentra, la descarga de Docker Hub, crea un contenedor a partir de ella y lo ejecuta. Lea con atención el mensaje que imprime, porque describe exactamente ese flujo.

Captura N.º 1: salida de `docker --version`, `docker compose version` y `docker run hello-world`. → regístrelo en la Hoja de Evidencias.

Pregunta N.º 1: ¿Qué diferencia hay entre el cliente de Docker (`docker`) y el demonio (`dockerd`)? ¿Por qué `docker info` falla si Docker Desktop está cerrado? → regístrelo en la Hoja de Evidencias.

Laboratorio 1 — Imágenes y ciclo de vida de un contenedor

Objetivo: dominar el ciclo de vida completo: buscar → descargar → ejecutar → inspeccionar → detener → eliminar.

Nota: Una IMAGEN es una plantilla de solo lectura (como una clase). Un CONTENEDOR es una instancia en ejecución de esa imagen (como un objeto). De una misma imagen se pueden crear muchos contenedores.

Paso 1. Buscar y descargar imágenes del registro

```
$ docker search nginx
$ docker pull nginx:alpine
$ docker pull mongo:7
$ docker images
```

Observe la columna TAG. La etiqueta alpine indica una variante basada en Alpine Linux, mucho más liviana. Compare el tamaño de nginx:alpine (~50 MB) contra nginx:latest (~200 MB).

Atención: Nunca use la etiqueta :latest en producción. No significa "la última versión estable", significa "la etiqueta por defecto", y cambia sin aviso. Fije siempre una versión: mongo:7, node:20-alpine.

Captura N.º 2: salida de docker images mostrando al menos nginx:alpine y mongo:7. → *regístrelo en la Hoja de Evidencias.*

Paso 2. Ejecutar un contenedor y publicar un puerto

```
$ docker run -d --name web1 -p 8080:80 nginx:alpine
$ docker ps
$ docker ps -a
```

Significado de cada bandera:

Bandera	Significado
-d	detached: el contenedor corre en segundo plano y devuelve el control a la terminal.
--name web1	Asigna un nombre legible. Sin esta bandera Docker genera un nombre aleatorio.
-p 8080:80	Publica el puerto: 8080 es el puerto del HOST, 80 es el puerto DENTRO del contenedor. Se lee HOST:CONTENEDOR.
nginx:alpine	Imagen y etiqueta a partir de la cual se crea el contenedor.

Abra en el navegador <http://localhost:8080> — debe ver la página de bienvenida de Nginx.

Captura N.º 3: navegador mostrando la página de Nginx en localhost:8080, junto con la salida de docker ps. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 2: En -p 8080:80, ¿qué pasaría si ejecuta un segundo contenedor con -p 8080:80? ¿Y con -p 8081:80? Pruébalo y explique el resultado. → *regístrelo en la Hoja de Evidencias.*

Paso 3. Inspeccionar el contenedor en ejecución

```
$ docker logs web1
$ docker logs -f web1          # sigue el log en vivo (Ctrl+C para salir)
$ docker inspect web1
$ docker stats --no-stream
$ docker top web1
```

docker inspect devuelve un JSON con toda la configuración del contenedor: red, volúmenes, variables de entorno, comando de arranque. Es la herramienta de diagnóstico principal cuando algo no funciona.

```
# Filtrar la IP del contenedor dentro de la red de Docker
$ docker inspect -f "{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}" web1
```

Paso 4. Entrar al contenedor

```
$ docker exec -it web1 sh
```

```
# ya dentro del contenedor:
/ # ls /usr/share/nginx/html
/ # cat /etc/os-release
/ # echo "<h1>Hola COM-600</h1>" > /usr/share/nginx/html/index.html
/ # exit
```

Recargue <http://localhost:8080>: la página cambió. Las banderas -it significan interactivo (-i) con terminal (-t). En imágenes Alpine se usa sh; en imágenes basadas en Debian/Ubuntu se usa bash.

Captura N.º 4: terminal dentro del contenedor mostrando cat /etc/os-release, y el navegador con la página modificada. → *regístrelo en la Hoja de Evidencias.*

Paso 5. Ciclo de vida: detener, iniciar, reiniciar, matar

```
$ docker stop web1      # envía SIGTERM y espera 10s (apagado ordenado)
$ docker ps             # ya no aparece
$ docker ps -a          # aparece con estado Exited
$ docker start web1
$ docker restart web1
$ docker kill web1      # envía SIGKILL: corte inmediato, sin apagado ordenado
```

Tip: stop es un apagado ordenado; kill es tirar del cable. En una aplicación real, stop permite cerrar conexiones a la base de datos y terminar las peticiones en curso. Use kill solo cuando el contenedor no responde.

Paso 6. Eliminar contenedores e imágenes

```
$ docker start web1
$ docker rm web1          # ERROR: no se puede borrar un contenedor en
ejecución
$ docker stop web1
$ docker rm web1
$ docker rm -f web1       # forzado: detiene y borra en un solo paso
$ docker ps -a           # confirme que ya no existe

$ docker rmi nginx:alpine # borra la IMAGEN
$ docker images
```

Captura N.º 5: secuencia del error al intentar borrar el contenedor en ejecución y su eliminación posterior. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 3: Al borrar el contenedor web1, ¿qué pasó con el archivo index.html que modificó en el Paso 4? Vuelva a crear un contenedor nginx y compruébelo. ¿Qué conclusión saca sobre dónde deben guardarse los datos? → *regístrelo en la Hoja de Evidencias.*

Paso 7. Contenedores efímeros y comandos de un solo uso

```
$ docker run --rm alpine echo "Hola desde un contenedor efímero"
$ docker run --rm -it alpine sh
$ docker run --rm alpine cat /etc/os-release
```

La bandera --rm elimina el contenedor automáticamente al terminar. Es la forma correcta de ejecutar tareas puntuales sin dejar basura en el sistema.

Paso 8. Historial y capas de una imagen

```
$ docker history mongo:7
$ docker image inspect mongo:7 --format "{{.Size}}"
```

Cada línea de docker history es una CAPA. Las imágenes de Docker son una pila de capas de solo lectura; el contenedor añade encima una capa escribible que se destruye al eliminarlo. Este concepto es la razón de todo lo que verá en los Laboratorios 2 y 3.

Captura N.º 6: salida de docker history mongo:7. → *regístrelo en la Hoja de Evidencias.*

Laboratorio 2 — Construcción de imágenes con Dockerfile

Objetivo: escribir un Dockerfile, construir una imagen propia y entender el cacheo de capas.

Paso 1. Crear la aplicación de ejemplo

Dentro de practica2-docker cree la carpeta api y dentro de ella los tres archivos siguientes.

api/package.json

```
{
  "name": "api-tareas",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

api/app.js

```
const express = require('express');
const os = require('os');

const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({
    mensaje: 'API de Tareas funcionando dentro de un contenedor',
    host: os.hostname(),
    version: process.env.APP_VERSION || '1.0.0'
  });
});

app.get('/salud', (req, res) => res.json({ estado: 'ok' }));

app.listen(PORT, () => console.log(`Servidor escuchando en el puerto ${PORT}`));
```

api/Dockerfile

```
# 1. Imagen base: Node 20 sobre Alpine Linux (liviana)
FROM node:20-alpine

# 2. Directorio de trabajo dentro del contenedor
WORKDIR /app

# 3. Copiar SOLO los manifiestos de dependencias (clave para el caché)
COPY package*.json ./
```

```
# 4. Instalar dependencias de producción
RUN npm install --omit=dev

# 5. Copiar el resto del código fuente
COPY . .

# 6. Documentar el puerto que expone la aplicación
EXPOSE 3000

# 7. Comando que se ejecuta al arrancar el contenedor
CMD ["node", "app.js"]
```

api/.dockerignore

```
node_modules
npm-debug.log
.git
.gitignore
.env
Dockerfile
.dockerignore
README.md
```

Nota: El archivo `.dockerignore` cumple el mismo papel que `.gitignore`: excluye archivos del contexto de construcción. Sin él, `node_modules` de su máquina se copiaría dentro de la imagen, la haría enorme y podría romperla (los binarios compilados de su sistema operativo no siempre sirven dentro de Alpine).

Paso 2. Construir la imagen

```
$ cd api
$ docker build -t api-tareas:1.0 .
$ docker images | findstr api-tareas      # Windows
$ docker images | grep api-tareas         # Linux / macOS
```

El punto final del comando es el **CONTEXT** de construcción: la carpeta que se envía al demonio de Docker. La bandera `-t` asigna nombre:etiqueta a la imagen resultante.

Captura N.º 7: salida completa de `docker build` mostrando los pasos numerados y el mensaje final de éxito. → *regístrelo en la Hoja de Evidencias.*

Paso 3. Ejecutar y probar su propia imagen

```
$ docker run -d --name api1 -p 3000:3000 api-tareas:1.0
$ docker logs api1
$ curl http://localhost:3000
$ curl http://localhost:3000/salud
```

Compare el campo `host` de la respuesta JSON con el ID del contenedor que muestra `docker ps`: son el mismo valor. Docker usa el ID corto del contenedor como `hostname`.

Captura N.º 8: respuesta JSON de `http://localhost:3000` en el navegador o con `curl`. → *regístrelo en la Hoja de Evidencias.*

Paso 4. Comprobar el caché de capas

Modifique únicamente el mensaje dentro de app.js (por ejemplo, agregue su nombre) y vuelva a construir:

```
$ docker build -t api-tareas:1.1 .
```

Observe la salida: los pasos hasta RUN npm install aparecen como CACHED y la construcción tarda segundos en lugar de minutos. Eso ocurre porque copiamos package.json ANTES que el código: si el código cambia pero las dependencias no, Docker reutiliza la capa de npm install.

Ahora invierta el orden en el Dockerfile a modo de experimento (COPY . . antes de RUN npm install), reconstruya y compare.

Captura N.º 9: las dos construcciones, mostrando las líneas CACHED en la primera y la reinstalación de dependencias en la segunda. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 4: ¿Por qué el orden de las instrucciones del Dockerfile afecta al tiempo de construcción? Explique la relación entre instrucción, capa y caché. → *regístrelo en la Hoja de Evidencias.*

Paso 5. Variables de entorno y etiquetado

```
$ docker run -d --name api2 -p 3001:3000 -e APP_VERSION=2.0-beta api-tareas:1.0
$ curl http://localhost:3001
```

```
# Etiquetar la misma imagen con otro nombre (no la duplica: es un alias)
$ docker tag api-tareas:1.0 api-tareas:estable
$ docker images
```

Tip: Las variables de entorno son el mecanismo estándar para configurar un contenedor sin reconstruir la imagen. Una misma imagen debe poder correr en desarrollo, pruebas y producción cambiando solo sus variables. Es el principio III de la metodología Twelve-Factor App.

Paso 6. Reducir el tamaño con multi-stage build (opcional pero recomendado)

Cree api/Dockerfile.multi con el siguiente contenido:

```
# ----- Etapa 1: construcción -----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev

# ----- Etapa 2: imagen final -----
FROM node:20-alpine
WORKDIR /app

# Ejecutar como usuario sin privilegios (buena práctica de seguridad)
RUN addgroup -S app && adduser -S app -G app

COPY --from=builder /app/node_modules ./node_modules
COPY . .

USER app
EXPOSE 3000
CMD ["node", "app.js"]

$ docker build -f Dockerfile.multi -t api-tareas:slim .
$ docker images | grep api-tareas
```

Captura N.º 10: comparación de tamaños entre api-tareas:1.0 y api-tareas:slim. → *regístrelo en la Hoja de Evidencias.*

Resumen de instrucciones del Dockerfile

Instrucción	Para qué sirve	Ejemplo
FROM	Imagen base sobre la que se construye. Siempre la primera.	FROM node:20-alpine
WORKDIR	Directorio de trabajo dentro del contenedor. Lo crea si no existe.	WORKDIR /app
COPY	Copia archivos del host a la imagen.	COPY . .
RUN	Ejecuta un comando DURANTE la construcción y genera una capa.	RUN npm install
ENV	Define una variable de entorno permanente en la imagen.	ENV NODE_ENV=production
EXPOSE	Documenta el puerto. No lo publica: eso lo hace -p.	EXPOSE 3000
USER	Usuario con el que se ejecutan las instrucciones siguientes.	USER app
CMD	Comando por defecto al arrancar. Se puede sobrescribir.	CMD ["node", "app.js"]
ENTRYPOINT	Comando fijo al arrancar. Los argumentos de CMD se le pasan.	ENTRYPOINT ["node"]

Atención: Diferencia clave: RUN se ejecuta al CONSTRUIR la imagen (una sola vez). CMD se ejecuta al ARRANCAR el contenedor (cada vez). Confundirlas es el error más frecuente al escribir un primer Dockerfile.

Laboratorio 3 — Volúmenes y persistencia de datos

Objetivo: comprobar que un contenedor es efímero y aprender a persistir datos con volúmenes nombrados y bind mounts.

Paso 1. Demostrar la pérdida de datos (contenedor sin volumen)

```
$ docker run -d --name mongo-sin-volumen -p 27017:27017 mongo:7
$ docker exec -it mongo-sin-volumen mongosh

# Ya dentro de mongosh:
use tareasdb
db.tareas.insertOne({ titulo: "Estudiar Docker", completada: false })
db.tareas.find()
exit
```

Ahora destruya el contenedor y vuelva a crearlo:

```
$ docker rm -f mongo-sin-volumen
$ docker run -d --name mongo-sin-volumen -p 27017:27017 mongo:7
```

```
$ docker exec -it mongo-sin-volumen mongosh --eval "use tareasdb; db.tareas.find()"
```

Atención: El resultado está vacío. Los datos vivían en la capa escribible del contenedor, que se destruye junto con él. Esta es la razón de existir de los volúmenes.

Captura N.º 11: el documento insertado antes de borrar el contenedor y la consulta vacía después de recrearlo. → *regístrelo en la Hoja de Evidencias.*

Paso 2. Volumen nombrado: persistencia real

```
$ docker volume create datos_mongo
$ docker volume ls
$ docker volume inspect datos_mongo

$ docker rm -f mongo-sin-volumen
$ docker run -d --name mongo-con-volumen -p 27017:27017 \
  -v datos_mongo:/data/db mongo:7
```

Nota: En Windows (CMD o PowerShell) escriba el comando en una sola línea, sin la barra invertida \ de continuación.

Inserte de nuevo un documento, destruya el contenedor y vuelva a crearlo con el mismo volumen:

```
$ docker exec -it mongo-con-volumen mongosh --eval
"db.getSiblingDB('tareasdb').tareas.insertOne({titulo:'Persistir datos',
completada:true})"

$ docker rm -f mongo-con-volumen
$ docker run -d --name mongo-con-volumen -p 27017:27017 -v datos_mongo:/data/db
mongo:7
$ docker exec -it mongo-con-volumen mongosh --eval
"db.getSiblingDB('tareasdb').tareas.find()"
```

Esta vez el documento sigue ahí. La ruta /data/db es donde MongoDB guarda sus archivos; al montar el volumen sobre esa ruta, los datos salen del ciclo de vida del contenedor.

Captura N.º 12: el documento recuperado después de destruir y recrear el contenedor con el volumen. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 5: ¿Dónde se almacena físicamente un volumen nombrado? Use `docker volume inspect datos_mongo` y explique el campo Mountpoint. ¿Por qué en Windows esa ruta no es accesible directamente desde el Explorador de archivos? → *regístrelo en la Hoja de Evidencias.*

Paso 3. Bind mount: montar una carpeta del host

Un bind mount enlaza una carpeta real de su máquina dentro del contenedor. Es la técnica que se usa en desarrollo para ver los cambios del código sin reconstruir la imagen.

```
# Cree una carpeta sitio con un archivo index.html
$ mkdir sitio
$ echo "<h1>Sitio montado desde el host</h1>" > sitio/index.html

# Linux / macOS
$ docker run -d --name web-dev -p 8080:80 -v $(pwd)/sitio:/usr/share/nginx/html
nginx:alpine

# Windows PowerShell
```

```
$ docker run -d --name web-dev -p 8080:80 -v ${PWD}/sitio:/usr/share/nginx/html
nginx:alpine

# Windows CMD
$ docker run -d --name web-dev -p 8080:80 -v %cd%/sitio:/usr/share/nginx/html
nginx:alpine
```

Abra `http://localhost:8080`, luego edite `sitio/index.html` en su editor y recargue el navegador: el cambio aparece de inmediato, sin reconstruir ni reiniciar nada.

Captura N.º 13: el navegador antes y después de editar `index.html` desde el host. → *regístrelo en la Hoja de Evidencias.*

Paso 4. Comparación de los tres tipos de almacenamiento

Tipo	Sintaxis	Gestionado por	Uso típico
Volumen nombrado	<code>-v nombre:/ruta</code>	Docker	Datos de bases de datos, archivos subidos. Es la opción recomendada en producción.
Bind mount	<code>-v /ruta/host:/ruta</code>	El sistema operativo del host	Desarrollo: código fuente, archivos de configuración.
tmpfs	<code>--tmpfs /ruta</code>	Memoria RAM	Datos temporales o sensibles que no deben tocar el disco.

Paso 5. Respaldo y restauración de un volumen

Un volumen se respalda montándolo en un contenedor efímero junto con una carpeta del host:

```
# Respaldo (PowerShell: reemplace $(pwd) por ${PWD})
$ docker run --rm -v datos_mongo:/data -v $(pwd):/backup alpine \
  tar czf /backup/backup_mongo.tar.gz -C /data .

# Restauración sobre un volumen nuevo
$ docker volume create datos_mongo_restaurado
$ docker run --rm -v datos_mongo_restaurado:/data -v $(pwd):/backup alpine \
  tar xzf /backup/backup_mongo.tar.gz -C /data
```

Captura N.º 14: el archivo `backup_mongo.tar.gz` creado en su carpeta y la restauración completada. → *regístrelo en la Hoja de Evidencias.*

Paso 6. Limpieza de volúmenes

```
$ docker volume ls
$ docker volume rm datos_mongo_restaurado
$ docker volume prune # borra TODOS los volúmenes no usados
```

Atención: `docker volume prune` borra datos de forma irreversible. Antes de ejecutarlo en cualquier entorno real, verifique con `docker volume ls` qué volúmenes quedarían sin contenedor asociado.

Laboratorio 4 — Redes en Docker

Objetivo: entender cómo se comunican los contenedores entre sí y por qué el nombre del servicio funciona como nombre de host.

Paso 1. Listar e inspeccionar las redes existentes

```
$ docker network ls
$ docker network inspect bridge
```

Docker crea tres redes por defecto:

Red	Driver	Comportamiento
bridge	bridge	Red por defecto. Los contenedores se ven por IP, pero NO por nombre.
host	host	El contenedor comparte la pila de red del host. No hay aislamiento ni mapeo de puertos.
none	null	Sin red. Aislamiento total.

Paso 2. Comprobar que en la red por defecto no hay DNS por nombre

```
$ docker run -d --name servidor nginx:alpine
$ docker run --rm -it alpine sh
/ # apk add --no-cache curl
/ # curl http://servidor          # FALLA: no se resuelve el nombre
/ # exit
```

Paso 3. Crear una red definida por el usuario

```
$ docker network create red_practica
$ docker network ls

$ docker rm -f servidor
$ docker run -d --name servidor --network red_practica nginx:alpine
$ docker run --rm -it --network red_practica alpine sh
/ # apk add --no-cache curl
/ # curl http://servidor          # FUNCIONA: responde el HTML de Nginx
/ # ping -c 3 servidor
/ # exit
```

Nota: En una red definida por el usuario, Docker levanta un servidor DNS interno que resuelve el NOMBRE del contenedor a su IP. Por eso en Docker Compose una API se conecta a su base de datos usando mongodb://mongo:27017 y no una dirección IP: "mongo" es el nombre del servicio. Este es el mecanismo de descubrimiento de servicios más simple en una arquitectura de microservicios.

Captura N.º 15: el curl fallando en la red por defecto y funcionando en red_practica. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 6: ¿Por qué es una mala idea que un microservicio se conecte a otro usando una dirección IP fija? Relacione su respuesta con lo que acaba de observar. → *regístrelo en la Hoja de Evidencias.*

Paso 4. Conectar y desconectar contenedores de una red

```
$ docker network inspect red_practica
$ docker network disconnect red_practica servidor
$ docker network connect red_practica servidor
$ docker rm -f servidor
$ docker network rm red_practica
```

Laboratorio 5 — Docker Compose: API Node.js + MongoDB

Objetivo: orquestar una aplicación multi-contenedor con un solo archivo declarativo, integrando todo lo anterior: imagen propia, volumen, red y variables de entorno.

Nota: Docker Compose reemplaza una lista larga de comandos docker run por un archivo YAML versionable. Es el paso previo a Kubernetes: mismos conceptos (servicios, volúmenes, redes), pero en una sola máquina.

Paso 1. Estructura de carpetas

```
practica2-docker/
├── docker-compose.yml
├── .env
├── .env.example
├── .gitignore
├── README.md
├── api/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── package.json
│   └── app.js
```

Paso 2. Actualizar la API para que use MongoDB

api/package.json

```
{
  "name": "api-tareas",
  "version": "2.0.0",
  "main": "app.js",
  "scripts": { "start": "node app.js" },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.5.0"
  }
}
```

api/app.js

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3000;
const MONGO_URL = process.env.MONGO_URL || 'mongodb://localhost:27017/tareasdb';

const Tarea = mongoose.model('Tarea', new mongoose.Schema({
  titulo: { type: String, required: true },
  completada: { type: Boolean, default: false }
}), { timestamps: true });

// Health check: lo usa Docker para saber si el servicio está sano
app.get('/salud', (req, res) => res.json({
  estado: 'ok',
  db: mongoose.connection.readyState === 1 ? 'conectada' : 'desconectada'
}));
```

```

    }));

app.get('/tareas', async (req, res) => res.json(await Tarea.find()));

app.post('/tareas', async (req, res) => {
  try {
    const tarea = await Tarea.create(req.body);
    res.status(201).json(tarea);
  } catch (e) {
    res.status(400).json({ error: e.message });
  }
});

app.put('/tareas/:id', async (req, res) => {
  const t = await Tarea.findByIdAndUpdate(req.params.id, req.body, { new: true });
  if (!t) return res.status(404).json({ error: 'Tarea no encontrada' });
  res.json(t);
});

app.delete('/tareas/:id', async (req, res) => {
  const t = await Tarea.findByIdAndDelete(req.params.id);
  if (!t) return res.status(404).json({ error: 'Tarea no encontrada' });
  res.status(204).send();
});

mongoose.connect(MONGO_URL)
  .then(() => {
    console.log('Conectado a MongoDB');
    app.listen(PORT, () => console.log(`API escuchando en el puerto ${PORT}`));
  })
  .catch(err => { console.error('Error de conexión:', err.message);
    process.exit(1); });

```

Paso 3. Archivo de variables de entorno

.env

```

API_PORT=3000
MONGO_DB=tareasdb
MONGOEXPRESS_PORT=8081

```

.gitignore

```

node_modules/
.env

```

Copie .env como .env.example dejando los valores vacíos o de ejemplo: ese sí se sube al repositorio.

Paso 4. Escribir el docker-compose.yml

```

services:

  api:
    build: ./api
    container_name: api_tareas
    ports:
      - "${API_PORT}:3000"
    environment:
      PORT: 3000
      MONGO_URL: mongodb://mongo:27017/${MONGO_DB}

```

```

    depends_on:
      mongo:
        condition: service_healthy
    networks:
      - red_practica
    restart: unless-stopped

mongo:
  image: mongo:7
  container_name: mongo_tareas
  volumes:
    - datos_mongo:/data/db
  networks:
    - red_practica
  healthcheck:
    test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 20s
    restart: unless-stopped

mongo-express:
  image: mongo-express:1.0.2
  container_name: mongo_ui
  ports:
    - "${MONGOEXPRESS_PORT}:8081"
  environment:
    ME_CONFIG_MONGODB_SERVER: mongo
    ME_CONFIG_BASICAUTH: "false"
  depends_on:
    - mongo
  networks:
    - red_practica

volumes:
  datos_mongo:

networks:
  red_practica:
    driver: bridge

```

Atención: El YAML es sensible a la indentación y NO admite tabuladores: use siempre 2 espacios. Un error de indentación es la causa del 90 % de los fallos al escribir un compose por primera vez. Valide con: `docker compose config`

Paso 5. Levantar el stack completo

```

$ docker compose config      # valida el YAML y muestra la configuración final
$ docker compose up -d --build
$ docker compose ps
$ docker compose logs -f api

```

Con un solo comando Docker construyó la imagen de la API, descargó MongoDB y mongo-express, creó la red `red_practica`, creó el volumen `datos_mongo` y arrancó los tres servicios en el orden correcto gracias al `healthcheck`.

Captura N.º 16: salida de `docker compose ps` con los tres servicios en estado `running/healthy`. → *regístrelo en la Hoja de Evidencias.*

Paso 6. Probar la API

```
$ curl http://localhost:3000/salud

$ curl -X POST http://localhost:3000/tareas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"'Terminar la practica 2'}'

$ curl http://localhost:3000/tareas
```

Abra también `http://localhost:8081` (mongo-express) y verifique que la base `tareasdb` y la colección `tareas` existen con el documento insertado.

Captura N.º 17: la respuesta del POST y la colección vista desde mongo-express en el navegador. → *regístrelo en la Hoja de Evidencias.*

Paso 7. Comprobar el DNS interno entre servicios

```
$ docker compose exec api sh
/app # ping -c 3 mongo
/app # env | grep MONGO
/app # exit
```

El nombre `mongo` se resuelve porque ambos servicios comparten la red `red_practica`. Ese es exactamente el mismo mecanismo que usará más adelante con Kubernetes.

Paso 8. Verificar la persistencia del stack

```
$ docker compose down          # detiene y borra contenedores y red, NO el
volumen
$ docker volume ls             # datos_mongo sigue existiendo
$ docker compose up -d
$ curl http://localhost:3000/tareas # los datos siguen ahí
```

Captura N.º 18: las tareas recuperadas después de un `down` y un `up`. → *regístrelo en la Hoja de Evidencias.*

Pregunta N.º 7: ¿Cuál es la diferencia entre `docker compose down` y `docker compose down -v`? Ejecute el segundo y describa qué ocurre con los datos. → *regístrelo en la Hoja de Evidencias.*

Paso 9. Escalado horizontal

Para escalar hay que quitar `container_name` y la publicación fija del puerto del servicio `api` (o usar un rango). Pruebe:

```
$ docker compose up -d --scale api=3
$ docker compose ps
```

Tip: Al escalar aparecen tres contenedores de la API. El DNS interno de Docker devuelve las tres IP en rotación, lo que da un balanceo de carga muy básico. Un balanceador real (Nginx, Traefik) o un orquestador (Kubernetes) resuelve esto correctamente: es el tema del Ejercicio 5 de la Parte 2.

Comandos esenciales de Docker Compose

Comando	Qué hace
<code>docker compose up -d</code>	Crea y arranca todos los servicios en segundo plano.

Comando	Qué hace
<code>docker compose up -d --build</code>	Igual, pero reconstruye las imágenes con build.
<code>docker compose ps</code>	Estado de los servicios del stack.
<code>docker compose logs -f <servicio></code>	Sigue los logs de un servicio en vivo.
<code>docker compose exec <servicio> sh</code>	Abre una terminal dentro de un servicio en ejecución.
<code>docker compose stop / start</code>	Detiene o arranca sin destruir los contenedores.
<code>docker compose down</code>	Detiene y elimina contenedores y redes. Conserva los volúmenes.
<code>docker compose down -v</code>	Igual que el anterior y ADEMÁS borra los volúmenes (pierde los datos).
<code>docker compose config</code>	Valida el YAML y muestra la configuración resuelta.
<code>docker compose restart <servicio></code>	Reinicia un único servicio.

Laboratorio 6 — Publicar la imagen en Docker Hub

Objetivo: distribuir la imagen construida para que pueda ejecutarse en cualquier otra máquina.

```
$ docker login

# La etiqueta DEBE tener el formato usuario/imagen:version
$ docker tag api-tareas:1.0 SU_USUARIO/api-tareas:1.0
$ docker push SU_USUARIO/api-tareas:1.0

# Comprobación: borre la imagen local y descárguela del registro
$ docker rmi SU_USUARIO/api-tareas:1.0
$ docker pull SU_USUARIO/api-tareas:1.0
$ docker run -d -p 3000:3000 SU_USUARIO/api-tareas:1.0
```

Captura N.º 19: la página del repositorio en hub.docker.com mostrando su imagen publicada. → regístrelo en la Hoja de Evidencias.

Atención: Nunca incluya credenciales, claves de API ni el archivo `.env` dentro de una imagen que vaya a publicar: cualquiera puede extraerlos con `docker history` o `docker save`.

Laboratorio 7 — Diagnóstico y limpieza del sistema

```
$ docker system df          # cuánto espacio ocupa Docker
$ docker system df -v       # detalle por imagen, contenedor y volumen

$ docker container prune    # borra contenedores detenidos
$ docker image prune        # borra imágenes huérfanas (<none>)
$ docker image prune -a     # borra TODAS las imágenes sin contenedor
$ docker builder prune      # limpia la caché de construcción

$ docker system prune       # limpieza general (sin volúmenes)
$ docker system prune -a --volumes # limpieza total: ¡destruye datos!
```

Captura N.º 20: salida de docker system df antes y después de la limpieza. → *regístrelo en la Hoja de Evidencias.*

Tabla resumen de comandos (chuleta)

Categoría	Comando	Descripción
Imágenes	<code>docker pull :<tag></code>	Descarga una imagen del registro.
	<code>docker images</code>	Lista las imágenes locales.
	<code>docker build -t <n>:<t> .</code>	Construye una imagen desde un Dockerfile.
	<code>docker tag <orig> <nuevo></code>	Crea un alias de una imagen.
	<code>docker rmi </code>	Elimina una imagen.
	<code>docker history </code>	Muestra las capas de la imagen.
Contenedores	<code>docker run -d -p H:C --name n </code>	Crea y arranca un contenedor.
	<code>docker ps / docker ps -a</code>	Lista contenedores activos / todos.
	<code>docker logs -f <c></code>	Muestra y sigue los logs.
	<code>docker exec -it <c> sh</code>	Abre una terminal dentro del contenedor.
	<code>docker stop / start / restart <c></code>	Controla el ciclo de vida.
	<code>docker rm -f <c></code>	Elimina el contenedor (forzado).
	<code>docker inspect <c></code>	Configuración completa en JSON.
Volúmenes	<code>docker volume create <v></code>	Crea un volumen nombrado.
	<code>docker run -v <v>:/ruta</code>	Monta un volumen en el contenedor.
	<code>docker volume ls / inspect / rm</code>	Lista, inspecciona y elimina volúmenes.
Redes	<code>docker network create <r></code>	Crea una red definida por el usuario.
	<code>docker run --network <r></code>	Conecta un contenedor a una red.
	<code>docker network ls / inspect</code>	Lista e inspecciona redes.
Compose	<code>docker compose up -d --build</code>	Levanta el stack completo.
	<code>docker compose down -v</code>	Derriba el stack y borra volúmenes.
Sistema	<code>docker system df / prune</code>	Uso de disco y limpieza.

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio debe entregarse con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Los ejercicios están ordenados por dificultad creciente. Los ejercicios 1 a 4 son obligatorios para todos. Los ejercicios 5 y 6 son los que marcan la diferencia entre una entrega correcta y una entrega sobresaliente.

Ejercicio 1. Dockerizar una aplicación propia (8 pts)

Tome una aplicación pequeña escrita por usted (puede ser la API REST de una práctica anterior, un script de Python, una aplicación de consola en Java o una página estática) y conviértala en una imagen de Docker.

Se pide:

- Un Dockerfile con al menos las instrucciones FROM, WORKDIR, COPY, RUN, EXPOSE y CMD, con un comentario explicando cada una.
- Un archivo .dockerignore justificado: explique en el informe qué excluye y por qué.
- La imagen etiquetada como <su-apellido>-app:1.0.
- Capturas de la construcción y de la aplicación funcionando dentro del contenedor.

Criterio de aceptación: la aplicación debe arrancar con un único comando docker run, sin instalar nada en el host.

Ejercicio 2. Optimización de una imagen (10 pts)

Se le entrega el siguiente Dockerfile, que funciona pero está mal escrito:

```
FROM node:20
COPY . /app
WORKDIR /app
RUN apt-get update
RUN apt-get install -y curl
RUN npm install
RUN npm install -g nodemon
ENV DB_PASSWORD=admin123
EXPOSE 3000
CMD npm start
```

Se pide:

- Identificar y explicar al menos CINCO problemas del Dockerfile anterior (tamaño, caché, seguridad, forma del CMD, etiquetado de la imagen base).
- Escribir una versión optimizada aplicando: imagen base alpine o slim, orden correcto para aprovechar el caché, unificación de instrucciones RUN, usuario sin privilegios, uso de la forma exec en CMD y eliminación de secretos.
- Construir ambas versiones y presentar una tabla comparativa con: tamaño de la imagen, número de capas y tiempo de reconstrucción tras modificar una línea del código.

Versión	Tamaño (MB)	N.º de capas	Tiempo de rebuild
Original			
Optimizada			
Mejora (%)			

Ejercicio 3. Persistencia, respaldo y restauración (10 pts)

Partiendo del stack del Laboratorio 5:

- Inserte al menos 10 tareas en la base de datos usando la API.
- Genere un respaldo del volumen datos_mongo en un archivo .tar.gz, siguiendo la técnica del contenedor efímero.
- Destruya el stack por completo, incluyendo volúmenes (docker compose down -v), y demuestre con una captura que los datos ya no existen.
- Restaure el respaldo sobre un volumen nuevo, levante el stack apuntando a él y demuestre que las 10 tareas volvieron.
- Escriba un script (respaldo.sh o respaldo.ps1) que automatice el respaldo con la fecha en el nombre del archivo.

En el informe explique: ¿por qué el respaldo se hace desde un contenedor efímero y no copiando la carpeta del volumen directamente desde el host?

Ejercicio 4. Aislamiento de red (8 pts)

Diseñe y monte con Docker Compose el siguiente escenario de tres servicios y dos redes:

Servicio	Red(es)	Debe poder	No debe poder
api	red_frontal, red_datos	Hablar con la base de datos y ser accesible desde el host	—
mongo	red_datos	Recibir conexiones de la API	Ser accesible desde el host ni desde el proxy
proxy (nginx)	red_frontal	Hablar con la API	Hablar con la base de datos

Se pide:

- El docker-compose.yml con las dos redes declaradas y sin publicar el puerto de MongoDB al host.
- Demostrar con capturas, desde dentro de los contenedores (docker compose exec ... ping / curl), que el proxy NO alcanza a mongo y que la API SÍ lo alcanza.
- Explicar por qué este diseño es más seguro que poner los tres servicios en una sola red.

Ejercicio 5. Stack con proxy inverso y balanceo (12 pts)

Amplíe el stack del Laboratorio 5 agregando Nginx como proxy inverso delante de la API:

- La API NO debe publicar puertos al host: el único punto de entrada es Nginx en el puerto 80.

- Escale la API a 3 réplicas (`docker compose up -d --scale api=3`) y configure Nginx con un bloque upstream para repartir las peticiones.
- Demuestre el balanceo: llame varias veces a la raíz de la API y muestre que el campo host de la respuesta JSON cambia entre las réplicas.
- Agregue healthcheck a los tres servicios y muestre `docker compose ps` con el estado healthy.
- Todas las credenciales y puertos deben venir de un archivo `.env`; entregue `.env.example` y verifique que `.env` está en el `.gitignore`.

Tip: Pista para la configuración de Nginx: dentro de la red de Compose, el nombre `api` resuelve a las IP de todas las réplicas. Un `proxy_pass http://api:3000;` con `resolver 127.0.0.11 valid=10s;` suele ser suficiente para observar la rotación.

Ejercicio 6. Proyecto integrador: stack completo documentado (12 pts)

Construya, desde cero, un stack de al menos tres servicios que resuelva un problema concreto elegido por usted (biblioteca, inventario, notas académicas, control de asistencia, etc.). Debe integrar todo lo aprendido:

Requisito	Debe incluir
Imagen propia	Al menos un servicio construido con Dockerfile propio (no solo imágenes oficiales).
Persistencia	Un volumen nombrado para la base de datos.
Red	Al menos una red definida por el usuario; comunicación entre servicios por nombre.
Configuración	Variables de entorno desde <code>.env</code> , sin credenciales en el repositorio.
Datos iniciales	Un mecanismo de carga de datos de prueba (script de seed o volumen de inicialización).
Salud	healthcheck en la base de datos y <code>depends_on</code> con <code>condition: service_healthy</code> .
Documentación	README.md con: descripción, requisitos, cómo levantar el stack, endpoints disponibles y cómo detenerlo.
Reproducibilidad	Un compañero debe poder clonar el repositorio y levantar todo con un solo comando.

Atención: Criterio de aceptación no negociable: el docente clonará su repositorio en una máquina limpia y ejecutará `docker compose up -d`. Si el stack no levanta con ese único comando, el ejercicio no se considera resuelto.

Preguntas de reflexión

Responda de forma breve y argumentada en el informe. Estas preguntas se evalúan por la calidad del razonamiento, no por la extensión.

- ¿Cuál es la diferencia entre una imagen y un contenedor? Explíquelo con una analogía propia (distinta a la de clase/objeto).

- ¿Por qué una máquina virtual pesa gigabytes y un contenedor megabytes, si ambos "aíslan" una aplicación?
- ¿Qué problema concreto resuelven las capas de una imagen y qué desventaja tienen?
- Si un contenedor es efímero, ¿cómo hacen las bases de datos en producción para no perder datos?
- ¿Cuándo usaría un bind mount y cuándo un volumen nombrado? Dé un ejemplo real de cada uno.
- ¿Por qué EXPOSE no publica un puerto? ¿Qué instrucción o bandera sí lo hace?
- ¿Qué diferencia hay entre CMD y ENTRYPOINT? Muestre un caso donde usar ENTRYPOINT sea claramente mejor.
- ¿Qué ventaja aporta depends_on con condition: service_healthy frente al depends_on simple?
- ¿Por qué guardar una contraseña con ENV dentro de un Dockerfile es una mala práctica de seguridad? ¿Qué alternativas existen?
- Relacione lo aprendido con la materia: ¿qué aporta Docker a una arquitectura de microservicios que un despliegue monolítico tradicional no tiene? Mencione al menos tres aportes.

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 20 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario/hostname).	15
Ejecución correcta	Las salidas de los comandos muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 7 preguntas de la Hoja de Evidencias están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. Dockerización	Dockerfile correcto y comentado, .dockerignore justificado, aplicación funcionando con un solo comando.	8
2. Optimización	Los 5 problemas identificados y explicados, versión optimizada funcional, tabla comparativa con datos reales.	10
3. Persistencia	Ciclo completo respaldo → destrucción → restauración demostrado con capturas, más el script automatizado.	10
4. Aislamiento de red	Dos redes correctamente declaradas, aislamiento demostrado desde dentro de los contenedores, justificación.	8
5. Proxy y balanceo	API sin puertos publicados, balanceo demostrado entre réplicas, healthchecks, gestión de .env.	12
6. Proyecto integrador	Stack levantado con un solo comando en máquina limpia, ocho requisitos cumplidos, README claro.	12
Subtotal		60

Penalizaciones

Situación	Descuento
Archivo .env con credenciales subido al repositorio	−5 pts
Uso de la etiqueta :latest en los archivos entregados	−3 pts

Situación	Descuento
node_modules subido al repositorio (falta .gitignore o .dockerignore)	-3 pts
Capturas que no corresponden a la máquina del estudiante o son de un compañero	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje / síntoma	Causa probable	Solución
Cannot connect to the Docker daemon	Docker Desktop no está iniciado.	Inicie Docker Desktop y espere a que el ícono deje de parpadear.
port is already allocated	Otro proceso o contenedor ya usa ese puerto del host.	Cambie el puerto del host (-p 8081:80) o libere el que está en uso.
no such file or directory (en COPY)	La ruta es relativa al contexto de construcción, no a su terminal.	Verifique desde qué carpeta ejecuta docker build y qué punto final le pasa.
ECONNREFUSED 127.0.0.1:27017	La API busca la base de datos en sí misma: dentro del contenedor, localhost es el propio contenedor.	Use el nombre del servicio: mongodb://mongo:27017
yaml: line N: did not find expected key	Indentación incorrecta o uso de tabuladores en el YAML.	Use 2 espacios, nunca tabuladores. Valide con docker compose config.
La API arranca antes que Mongo y muere	depends_on simple solo espera a que el contenedor arranque, no a que el servicio esté listo.	Agregue healthcheck a mongo y condition: service_healthy, más restart: unless-stopped.
Los cambios del código no se ven	La imagen tiene el código copiado dentro y no se reconstruyó.	docker compose up -d --build, o use un bind mount en desarrollo.
Se acabó el espacio en disco	Imágenes, caché de construcción y volúmenes acumulados.	docker system df para ver el detalle y docker system prune para limpiar.
Permiso denegado al escribir en un volumen	El contenedor corre con un usuario sin permisos sobre la carpeta montada.	Revise la instrucción USER y los permisos de la carpeta del host.

Tip: Antes de pedir ayuda, ejecute siempre estos tres comandos y lea su salida: `docker compose ps` (¿está corriendo?), `docker compose logs <servicio>` (¿qué dice el error?) y `docker inspect <contenedor>` (¿está configurado como usted cree?). El 80 % de los problemas se resuelve ahí.

Referencias

- Documentación oficial de Docker — <https://docs.docker.com>
- Referencia del Dockerfile — <https://docs.docker.com/reference/dockerfile/>
- Especificación de Docker Compose — <https://docs.docker.com/compose/compose-file/>
- Buenas prácticas para escribir Dockerfiles — <https://docs.docker.com/build/building/best-practices/>
- Docker Hub — <https://hub.docker.com>
- Newman, S. Building Microservices. 2.ª ed. O'Reilly, 2021 — Capítulo sobre despliegue y contenedores.